Physical design:

```
CREATE DATABASE InstrumentSelectionRecommendationSystem;

USE InstrumentSelectionRecommendationSystem;

CREATE TABLE AUDIT_LOG (
  log_id INT PRIMARY KEY IDENTITY(1,1),
  event_type VARCHAR(50) NOT NULL,
  description VARCHAR(MAX) NOT NULL,
  created_at DATETIME DEFAULT GETDATE()
);
GO


CREATE TABLE CATEGORY (
  category_id INT PRIMARY KEY IDENTITY(1,1),
  category_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE MUSICAL_BACKGROUND (
  background_id INT PRIMARY KEY IDENTITY(1,1),
  experience_level VARCHAR(20) NOT NULL CHECK (experience_level IN ('Beginner',
'Intermediate', 'Advanced'))
);

CREATE TABLE PERSONALITY (
  personality_id INT PRIMARY KEY IDENTITY(1,1),
  personality_type VARCHAR(50) NOT NULL,
  details VARCHAR(MAX)
);

CREATE TABLE GENRE (
  genre_id INT PRIMARY KEY IDENTITY(1,1),
  genre_name VARCHAR(50) NOT NULL UNIQUE
);


CREATE TABLE [USER] (
  user_id INT PRIMARY KEY IDENTITY(1,1),
  password VARCHAR(255) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  personality_id INT NULL,
  background_id INT NULL,
  FOREIGN KEY (personality_id) REFERENCES PERSONALITY(personality_id) ON DELETE SET
NULL,
```

```
  FOREIGN KEY (background_id) REFERENCES MUSICAL_BACKGROUND(background_id) ON
DELETE SET NULL
);

CREATE TABLE INSTRUMENT (
  instrument_id INT PRIMARY KEY IDENTITY(1,1),
  instrument_name VARCHAR(100) NOT NULL UNIQUE,
  type VARCHAR(50),
  difficulty_level VARCHAR(20) CHECK (difficulty_level IN ('Easy', 'Medium', 'Hard')),
  description VARCHAR(MAX),
  category_id INT NOT NULL,
  FOREIGN KEY (category_id) REFERENCES CATEGORY(category_id) ON DELETE CASCADE
);

CREATE TABLE RECOMMENDATION (
  recommendation_id INT PRIMARY KEY IDENTITY(1,1),
  match_score DECIMAL(5,2) NOT NULL,
  ranking INT NOT NULL,
  user_id INT NOT NULL,
  FOREIGN KEY (user_id) REFERENCES [USER](user_id) ON DELETE CASCADE
);



CREATE TABLE USER_GENRE (
  user_id INT NOT NULL,
  genre_id INT NOT NULL,
  PRIMARY KEY (user_id, genre_id),
  FOREIGN KEY (user_id) REFERENCES [USER](user_id) ON DELETE CASCADE,
  FOREIGN KEY (genre_id) REFERENCES GENRE(genre_id) ON DELETE CASCADE
);

CREATE TABLE INSTRUMENT_GENRE (
  instrument_id INT NOT NULL,
  genre_id INT NOT NULL,
  PRIMARY KEY (instrument_id, genre_id),
  FOREIGN KEY (instrument_id) REFERENCES INSTRUMENT(instrument_id) ON DELETE
CASCADE,
  FOREIGN KEY (genre_id) REFERENCES GENRE(genre_id) ON DELETE CASCADE
);

CREATE TABLE RECOMMENDATION_INSTRUMENT (
  recommendation_id INT NOT NULL,
  instrument_id INT NOT NULL,
  PRIMARY KEY (recommendation_id, instrument_id),
  FOREIGN KEY (recommendation_id) REFERENCES RECOMMENDATION(recommendation_id)
ON DELETE CASCADE,
```

```
  FOREIGN KEY (instrument_id) REFERENCES INSTRUMENT(instrument_id) ON DELETE
CASCADE
);
GO


CREATE TRIGGER trg_UserRegistrationAudit
ON [USER]
AFTER INSERT
AS
BEGIN
  SET NOCOUNT ON;

  INSERT INTO AUDIT_LOG (event_type, description)
  SELECT
    'USER_REGISTERED',
    CONCAT('New user account created: ', email, ' (User ID: ', user_id, ')')
  FROM inserted;
END;
GO

CREATE TRIGGER trg_ValidateRecommendationScore
ON RECOMMENDATION
INSTEAD OF INSERT
AS
BEGIN
  SET NOCOUNT ON;

  IF EXISTS (SELECT 1 FROM inserted WHERE match_score < 0.00 OR match_score > 100.00)
  BEGIN
    RAISERROR('Invalid Match Score: Score must be between 0.00 and 100.00.', 16, 1);
    RETURN;
  END

  INSERT INTO RECOMMENDATION (match_score, ranking, user_id)
  SELECT match_score, ranking, user_id FROM inserted;
END;
GO

CREATE PROCEDURE sp_RegisterUser
  @Email VARCHAR(100),
  @Password VARCHAR(255),
  @FirstName VARCHAR(50),
  @LastName VARCHAR(50),
  @PersonalityID INT = NULL,
  @BackgroundID INT = NULL
AS
BEGIN
```

```
  SET NOCOUNT ON;

  BEGIN TRY
    BEGIN TRANSACTION;

    IF EXISTS (SELECT 1 FROM [USER] WHERE email = @Email)
    BEGIN
      RAISERROR('Account registration failed: Email address is already registered.', 16, 1);
    END

    INSERT INTO [USER] (email, password, first_name, last_name, personality_id,
background_id)
    VALUES (@Email, @Password, @FirstName, @LastName, @PersonalityID,
@BackgroundID);

    DECLARE @NewUserID INT = SCOPE_IDENTITY();

    COMMIT TRANSACTION;

    SELECT @NewUserID AS new_user_id;
  END TRY
  BEGIN CATCH

    IF XACT_STATE() <> 0
    BEGIN
      ROLLBACK TRANSACTION;
    END

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrorState INT = ERROR_STATE();

    RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
  END CATCH
END;
GO


CREATE PROCEDURE sp_SaveUserGenrePreference
  @UserID INT,
  @GenreID INT
AS
BEGIN
  SET NOCOUNT ON;

  BEGIN TRY
    BEGIN TRANSACTION;
```

```
    IF NOT EXISTS (SELECT 1 FROM USER_GENRE WHERE user_id = @UserID AND genre_id =
@GenreID)
    BEGIN
      INSERT INTO USER_GENRE (user_id, genre_id)
      VALUES (@UserID, @GenreID);
    END

    COMMIT TRANSACTION;
  END TRY
  BEGIN CATCH
    IF XACT_STATE() <> 0
    BEGIN
      ROLLBACK TRANSACTION;
    END

    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    RAISERROR(@ErrorMessage, 16, 1);
  END CATCH
END;
GO


CREATE PROCEDURE sp_GenerateUserRecommendations
  @UserID INT
AS
BEGIN
  SET NOCOUNT ON;

  BEGIN TRY
    SELECT DISTINCT
      i.instrument_id,
      i.instrument_name,
      i.type,
      i.difficulty_level,
      c.category_name,
      i.description
    FROM INSTRUMENT i
    INNER JOIN CATEGORY c ON i.category_id = c.category_id
    INNER JOIN INSTRUMENT_GENRE ig ON i.instrument_id = ig.instrument_id
    INNER JOIN USER_GENRE ug ON ig.genre_id = ug.genre_id
    WHERE ug.user_id = @UserID;
  END TRY
  BEGIN CATCH
    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    RAISERROR(@ErrorMessage, 16, 1);
  END CATCH
END;
GO
```